

# Calculadora Distribuida: Primera Entrega

Universidad Panamericana

Roberto Requejo Fernández 0236851

A distributed system is a set of computers that work together to achieve a common goal. Instead of one computer performing all the operations, it shares and delegates tasks over the network so that other computers in the system perform a portion of them, thus avoiding concentration on a single device; it's a decentralized system. There are many advantages to using distributed systems, and the advantages are as follows:

- **Communication:** Communication is very effective and fast, as sockets are used to establish a tunnel through the network between connected devices to coordinate tasks effectively.
- **Shared resources:** Since all devices are interconnected, memory and processing power become shared, making more effective use of resources.
- **Fault tolerance:** If one device fails, the others can continue working without affecting the final user.

For this project, we had to create a distributed calculator, which consists of three programs. The first one is the client, responsible for displaying the graphical interface to the user so they can input operations and view the results. The second is a MOM (Message Oriented Middleware)

server that acts as an intermediary layer between the client and the server. And the third is the server, which is a program that receives information about the operations to perform, solves them, and sends the results back to the client.

When undertaking this project, the first thing I did was to make it work through the console, and then I proceeded to develop the graphical interface. This was a mistake because when implementing the graphical interface, the thread handling changed significantly, and I had to make several adjustments. However, I gained a good understanding of how this system works. The client was created using JavaFX and is quite simple; it includes a calculator and a section where the results of the operation results are accumulated.

The client connects via socket to the MOM server through the local network and connects to the port 6666. The client looks like the following image:

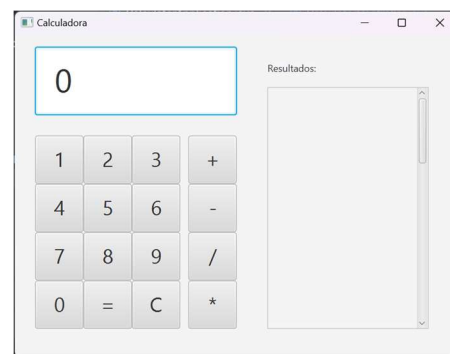


Fig 1. Calculator interface

I also developed a protocol for each of the three programs to perform their respective functions without unnecessary overhead. In the case of the client, when the user presses the equals button, it generates a string like this: "100,2,5,\*" This packet signifies the following: the "100" represents the packet code, which can be processed by the server and indicates that it's an operation to be

performed. Following that is the user-entered operation; in this case, the entered operation was "2\*5". Once the server resolves this operation, it will send a packet like the following: "200,2,5,\*,10". The "200" indicates that it's a packet for a successfully resolved operation, and only the result is appended to it. Since it has a code of 200, the operation is added to the results column.

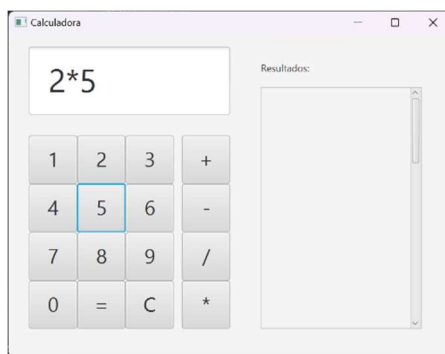


Fig 2. Example of user sending a request

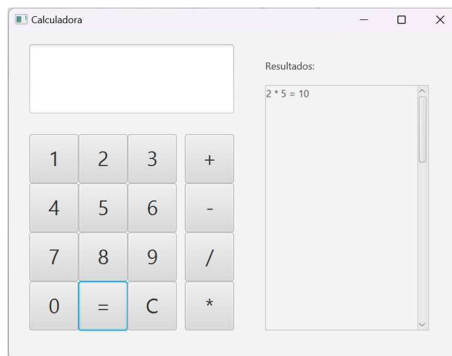


Fig 3. Example of response in the client

Then I created the MOM server, which functions in a very simple manner. All users, whether they are servers or clients, connect to it through sockets on port 6666, and it listens to everyone connected. Once it receives a packet from someone, it sends the same packet it received to all connected users, including the user who sent the

packet. This method of communication is called broadcast. That's why it's important to use the protocol I explained in the client so that each one performs its designated tasks. Below is a diagram of how the MOM server works.

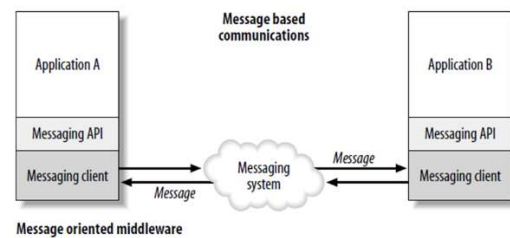


Fig 4. Application Flow Diagram

In this case Application A is our client and Application B is the server. Everything is connected via the MOM server that broadcasts the messages between the network.

Finally, I created the server, which is responsible for performing the operations and sending them to the user. As I explained in the client, we handle this through a protocol I explained. The server will receive a string in this format: "100,2,5,\*". The first number, which is "100," indicates that it's an operation to be performed, the "2" and "5" are the operands of the operation, and the last part is the operator. The server will receive this packet, solve the operation, and generate a new packet like this: "200,2,5,\*,10". It will change the packet code from 100 to 200, indicating that the operation was successfully executed, and it will add the result at the end of the packet, in this case, "10".

In summary, it's a distributed system with a significant room for improvement. In this

case, operations are duplicated depending on the number of servers and clients we have, and among clients, we receive operations that we never sent but were sent by another client. Another system or protocol needs to be implemented to prevent this from happening, regardless of the number of servers or clients. The end user should only see the requested operation once.

I also learned a lot about sockets. I had previously attempted to work with sockets without success, but Java is well-designed for socket usage and makes it appear quite straightforward with its library. Additionally, its handling of threads is also relatively straightforward.